

Bayesian Machine Learning in Quantitative Finance

Chapter 11: Calibrating the Nelson-Siegel Model

Based on Mongwe, Mbuva & Marwala

Springer, 2025

Chapter Overview

- 1 11.1 Introduction
- 2 11.2 The Nelson-Siegel Model
- 3 11.3 Advanced Hamiltonian Monte Carlo Methods
 - 11.3.1 HMC and the No-U-Turn Sampler
 - 11.3.2 Separable Shadow Hamiltonian Hybrid Monte Carlo
- 4 Toy Illustration: Bayesian NS Calibration with HMC
- 5 11.4 Numerical Experiments
 - 11.4.1 Data Description and Experiment Setup
 - 11.4.2 Performance Metrics
- 6 11.5 Results and Discussions
- 7 11.6 Conclusion

Outline

- 1 11.1 Introduction
- 2 11.2 The Nelson-Siegel Model
- 3 11.3 Advanced Hamiltonian Monte Carlo Methods
 - 11.3.1 HMC and the No-U-Turn Sampler
 - 11.3.2 Separable Shadow Hamiltonian Hybrid Monte Carlo
- 4 Toy Illustration: Bayesian NS Calibration with HMC
- 5 11.4 Numerical Experiments
 - 11.4.1 Data Description and Experiment Setup
 - 11.4.2 Performance Metrics
- 6 11.5 Results and Discussions
- 7 11.6 Conclusion

What Is a Yield Curve, and Why Model It? I

Intuition first. A *yield curve* (a.k.a. the term structure of interest rates) is simply the relationship between a bond's interest rate (yield) and its time to maturity. Plot yield on the vertical axis and maturity on the horizontal axis, and you get a snapshot of what the market currently believes about future interest rates: is short-term borrowing cheaper or more expensive than long-term borrowing right now? Central banks, bond traders, and corporate treasurers all watch this curve closely.

Why Bother with a Parametric Model?

A yield curve is observed only at a handful of maturities (e.g. 3-month, 2-year, 10-year, 30-year bonds). To price an arbitrary contract, or to smooth out noisy market quotes, we need a *continuous curve* that interpolates and extrapolates sensibly. Two broad modelling philosophies exist:

- **No-arbitrage models** (e.g. Hull–White): force the model to exactly match the curve at a point in time, crucial for pricing interest-rate derivatives (swaps, swaptions, caps).
- **Equilibrium models** (e.g. Vasicek, Cox–Ingersoll–Ross): model the dynamics of the instantaneous short rate; yields at other maturities follow from assumptions about the risk premium.

What Is a Yield Curve, and Why Model It? II

The Nelson–Siegel (NS) Model, Plainly

The **Nelson and Siegel (1987)** model takes a third route: it is a simple, parsimonious *curve-fitting* model with only 3–4 parameters representing **level**, **slope**, **curvature**, plus a **decay rate** λ . Despite its simplicity it is flexible enough to match almost every realistically-shaped yield curve (upward-sloping, inverted, humped), and it is exactly the kind of model that is easy to explain to a non-technical stakeholder (“level”, “slope” and “hump” are intuitive words). This is why central banks in Belgium, Finland, France, Germany, Italy, Norway, and Switzerland, as well as the European Central Bank (via the related Svensson extension), use NS-type models operationally to estimate zero-coupon yield curves.

What Is a Yield Curve, and Why Model It? III

The Calibration Problem

Calibrating the NS model means: given observed market yields at a set of maturities, find parameter values so the model's yield curve matches the data as closely as possible. This sounds like a simple regression, but in practice: (i) the objective function is nonlinear in the decay parameter λ and has multiple local optima, and (ii) the parameters are highly *collinear* — small data changes can cause wild, unstable jumps in the fitted parameters. The traditional fix is Ordinary Least Squares (OLS) or ridge regression on a grid of λ values. This chapter instead treats calibration as a **Bayesian inference** problem: we compute a full *posterior distribution* over the NS parameters, giving us both point estimates and honest uncertainty quantification, using advanced Hamiltonian-dynamics-based MCMC samplers — HMC, the No-U-Turn Sampler (NUTS), and Separable Shadow Hamiltonian Hybrid Monte Carlo (S2HMC).

Chapter Roadmap

11.2 the NS model itself → **11.3** HMC / NUTS (recap) and S2HMC (new) → **11.4** experiment setup (simulated NS data, ZAR swap curve, USA corporate bond curves) → **11.5** results → **11.6** conclusion.

Outline

- 1 11.1 Introduction
- 2 11.2 The Nelson-Siegel Model
- 3 11.3 Advanced Hamiltonian Monte Carlo Methods
 - 11.3.1 HMC and the No-U-Turn Sampler
 - 11.3.2 Separable Shadow Hamiltonian Hybrid Monte Carlo
- 4 Toy Illustration: Bayesian NS Calibration with HMC
- 5 11.4 Numerical Experiments
 - 11.4.1 Data Description and Experiment Setup
 - 11.4.2 Performance Metrics
- 6 11.5 Results and Discussions
- 7 11.6 Conclusion

The Nelson-Siegel Forward Rate and Yield Curve

Intuition. The NS model starts from an assumption about the *instantaneous forward rate* $f_t(\tau)$ — the interest rate you would lock in today (t) for a very short loan starting at time τ in the future. It is built from three exponential building blocks: a constant, a decaying exponential, and a decaying exponential multiplied by τ (which first rises, then decays — producing a hump).

Nelson-Siegel Forward Rate (Eq. 11.1)

$$f_t(\tau) = \beta_0 + \beta_1 \exp(-\lambda\tau) + \beta_2 \lambda\tau \exp(-\lambda\tau)$$

where $\{\beta_0, \beta_1, \beta_2, \lambda\}$ are the parameters of the model, t is the current (valuation) time, and τ is the time period between now and the point of interest on the curve. We require $\beta_0 > 0$ and $\lambda > 0$ for economic sensibility (the long end of the curve must be positive, and the decay must be a genuine decay).

The Nelson-Siegel Yield Curve

Because the *yield* to maturity is (by definition) the average of the forward rates out to that maturity, i.e. $y_t(\tau) = \frac{1}{\tau} \int_0^\tau f_t(s) ds$, integrating Eq. 11.1 gives the practically-used NS *yield curve*:

Nelson-Siegel Yield Curve (Eq. 11.2)

$$y_t(\tau) = \beta_0 + \beta_1 \left(\frac{1 - \exp(-\lambda\tau)}{\lambda\tau} \right) + \beta_2 \left(\frac{1 - \exp(-\lambda\tau)}{\lambda\tau} - \exp(-\lambda\tau) \right)$$

This is the equation actually fitted to market data. It is a *three-component exponential approximation* that works remarkably well in practice (Diebold & Li, 2006).

The Three Factor Loadings, One at a Time

The Three Factor Loadings, One at a Time

Write $y_t(\tau) = \beta_0 \cdot 1 + \beta_1 \cdot \ell_1(\tau) + \beta_2 \cdot \ell_2(\tau)$ where the *loadings* are functions of maturity τ and decay λ only:

- **Level loading** on β_0 is the constant 1 — it does not decay to zero as $\tau \rightarrow \infty$, so β_0 is the **long-term factor**. Changing β_0 shifts the *entire* curve up or down in parallel: a *parallel shift*.
- **Slope loading** $\ell_1(\tau) = \frac{1 - \exp(-\lambda\tau)}{\lambda\tau}$ starts at 1 when $\tau \rightarrow 0$ and decays monotonically and quickly to 0 as $\tau \rightarrow \infty$. So β_1 mainly moves the **short end** of the curve — it controls the **steepness** (slope) between short and long maturities.
- **Curvature loading** $\ell_2(\tau) = \frac{1 - \exp(-\lambda\tau)}{\lambda\tau} - \exp(-\lambda\tau)$ starts at 0, rises, and then converges back to 0 at long maturities — it is zero at both ends and peaks at intermediate maturities. So β_2 controls the **hump** (curvature) in the middle of the curve, and is a **medium-term factor**.

Diebold and Li (2006) show these three loadings can be read directly as **level, slope, and curvature** of the yield curve, and that this simple model is highly effective for in-sample fitting and out-of-sample forecasting.

Worked Numeric Example I

Worked Numeric Example II

Worked Numeric Example

Let $\beta_0 = 0.04$, $\beta_1 = -0.05$, $\beta_2 = 0.20$, $\lambda = 1.3$ (these are exactly the values used later in the book's simulated experiment, Sec. 11.4.1). Compute the yield at three maturities.

At $\tau = 1$ year: $\lambda\tau = 1.3$, $\exp(-1.3) = 0.2725$.

$$\ell_1(1) = \frac{1 - 0.2725}{1.3} = 0.5596, \quad \ell_2(1) = 0.5596 - 0.2725 = 0.2871$$

$$y_t(1) = 0.04 + (-0.05)(0.5596) + (0.20)(0.2871) = 0.04 - 0.0280 + 0.0574 = 0.0694 \Rightarrow 6.94\%$$

At $\tau = 5$ years: $\lambda\tau = 6.5$, $\exp(-6.5) \approx 0.0015$.

$$\ell_1(5) = 0.1536, \quad \ell_2(5) = 0.1521$$

$$y_t(5) = 0.04 - 0.05(0.1536) + 0.20(0.1521) = 0.04 - 0.00768 + 0.03042 = 0.0627 \Rightarrow 6.27\%$$

At $\tau = 20$ years: $\lambda\tau = 26$, $\exp(-26) \approx 0$.

$$\ell_1(20) \approx \frac{1}{26} = 0.0385, \quad \ell_2(20) \approx 0.0385$$

Priors in the Bayesian Formulation

Priors in the Bayesian Formulation

To calibrate NS in a Bayesian way we need priors on $\{\beta_0, \beta_1, \beta_2, \lambda\}$. Because $\beta_0 > 0$ and $\lambda > 0$ are required for the model to make economic sense, the book uses:

- **Gaussian** priors for β_1 and β_2 (these can be positive or negative — a positive or negative slope/hump are both plausible).
- **LogNormal** priors for β_0 and λ (these must stay positive).

This is analogous to the regularization that ridge regression imposes on OLS-based NS calibration (Annaert et al., 2013), but with the added benefit of a full posterior distribution rather than a single point estimate.

Figure `fig_ns_loadings.pdf` (generated from scratch, see the code on the next slide) shows all three loading curves for $\lambda = 1.3$: the level loading is flat at 1, the slope loading decays from 1 to 0, and the curvature loading rises from 0 to a hump around $\tau \approx 1/\lambda$ years before decaying back to 0.

Python: Nelson-Siegel Yield Curve and Loadings

Listing 1: NS yield curve, from scratch

```
1 import numpy as np
2
3 def loadings(tau, lam):
4     """Level, slope, curvature loadings."""
5     lt = lam * tau
6     level = np.ones_like(tau)
7     slope = (1 - np.exp(-lt)) / lt
8     curvature = slope - np.exp(-lt)
9     return level, slope, curvature
10
11 def ns_yield(tau, b0, b1, b2, lam):
12     """Nelson-Siegel yield  $y_t(\tau)$ , Eq. 11.2."""
13     lvl, slp, curv = loadings(tau, lam)
14     return b0 * lvl + b1 * slp + b2 * curv
15
16 # Worked numeric example from the slide
17 tau = np.array([1.0, 5.0, 20.0])
18 y = ns_yield(tau, b0=0.04, b1=-0.05,
19              b2=0.20, lam=1.3)
20 print(100 * y)
21 # -> [6.94  6.27  4.58] (percent)
```

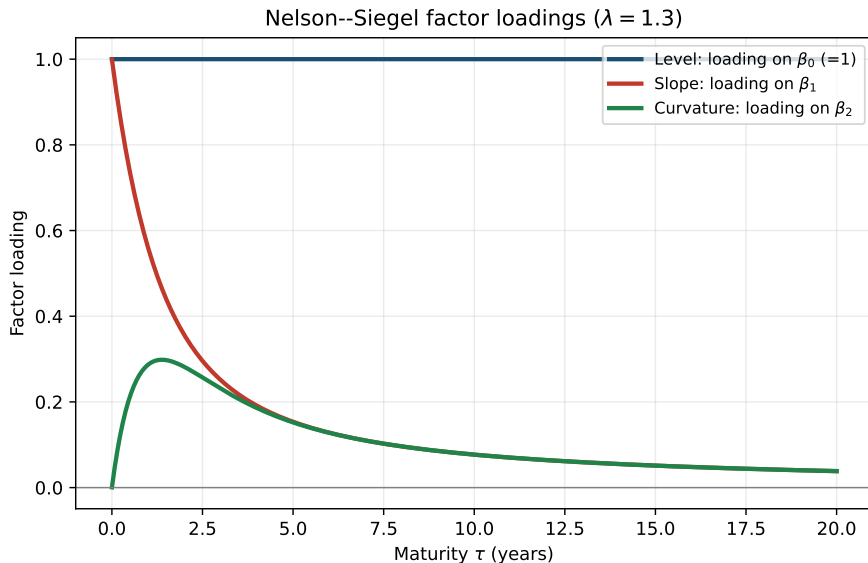
What This Verifies

Running this snippet reproduces exactly the hand-computed yields from the worked example: 6.94% at $\tau = 1$, 6.27% at $\tau = 5$, and 4.58% at $\tau = 20$ years.

Note

`loadings()` is exactly the triple $(1, \ell_1(\tau), \ell_2(\tau))$ used to build `fig_ns_loadings.pdf`. The same function is reused below to build the likelihood for Bayesian calibration via HMC/S2HMC.

Figure: The Three NS Factor Loading Curves



Outline

- 1 11.1 Introduction
- 2 11.2 The Nelson-Siegel Model
- 3 11.3 Advanced Hamiltonian Monte Carlo Methods
 - 11.3.1 HMC and the No-U-Turn Sampler
 - 11.3.2 Separable Shadow Hamiltonian Hybrid Monte Carlo
- 4 Toy Illustration: Bayesian NS Calibration with HMC
- 5 11.4 Numerical Experiments
 - 11.4.1 Data Description and Experiment Setup
 - 11.4.2 Performance Metrics
- 6 11.5 Results and Discussions
- 7 11.6 Conclusion

Recap: Hamiltonian Monte Carlo (HMC) and NUTS I

You have already seen HMC, Shadow HMC, and NUTS earlier in this lecture series (Chapters 3, 7, 9). Here is a fast, efficient recap of the essentials before we specialize to the NS-model calibration problem.

Intuition

Plain random-walk Metropolis proposes a new parameter value by adding small random noise — inefficient in high dimensions or on correlated posteriors. HMC instead treats the (negative-log) posterior as a *potential energy* surface, introduces an auxiliary *momentum* variable, and simulates a physical particle sliding over that surface. This uses gradient information to make large, informed proposals that are still likely to be accepted, dramatically reducing autocorrelation compared to random-walk methods.

Recap: Hamiltonian Monte Carlo (HMC) and NUTS II

Hamiltonian Dynamics (Eq. 11.3)

$$\frac{\partial \mathbf{w}}{\partial t} = \frac{\partial H(\mathbf{w}, \mathbf{p})}{\partial \mathbf{p}}; \quad \frac{\partial \mathbf{p}}{\partial t} = -\frac{\partial H(\mathbf{w}, \mathbf{p})}{\partial \mathbf{w}}$$

where $\mathbf{w}$ are the model parameters (here, the NS parameters), $\mathbf{p}$ is an auxiliary momentum variable of the same dimension as $\mathbf{w}$, and $H(\mathbf{w}, \mathbf{p}) = U(\mathbf{w}) + K(\mathbf{p})$ is the *Hamiltonian* (total energy): $U(\mathbf{w})$ is the potential energy (the negative log posterior of the NS parameters given yield data) and $K(\mathbf{p})$ is the kinetic energy (typically $K(\mathbf{p}) = \frac{1}{2}\mathbf{p}^T \mathbf{M}^{-1} \mathbf{p}$ for a Gaussian momentum distribution with mass/pre-conditioning matrix $\mathbf{M}$).

Recap: Hamiltonian Monte Carlo (HMC) and NUTS III

Leapfrog Integrator (Eq. 11.4)

HMC simulates these dynamics numerically using the *leapfrog* scheme, which alternates half-step momentum updates with full-step position updates:

$$\mathbf{p}_{t+\frac{\epsilon}{2}} = \mathbf{p}_t + \frac{\epsilon}{2} \frac{\partial H(\mathbf{w}_t, \mathbf{p}_t)}{\partial \mathbf{w}}, \quad \mathbf{w}_{t+\epsilon} = \mathbf{w}_t + \epsilon \mathbf{M}^{-1} \mathbf{p}_{t+\frac{\epsilon}{2}}$$

$$\mathbf{p}_{t+\epsilon} = \mathbf{p}_{t+\frac{\epsilon}{2}} + \frac{\epsilon}{2} \frac{\partial H(\mathbf{w}_{t+\epsilon}, \mathbf{p}_{t+\frac{\epsilon}{2}})}{\partial \mathbf{w}}$$

where ϵ is the discretization step size. Leapfrog is *symplectic* (volume- and, approximately, energy-preserving to $\mathcal{O}(\epsilon^2)$), which is exactly why HMC proposals are accepted far more often than naive Euler integration would allow.

Recap: Hamiltonian Monte Carlo (HMC) and NUTS IV

Metropolis Correction (Eq. 11.5)

Because leapfrog only approximately conserves H , a Metropolis-Hastings accept/reject step corrects for the residual discretization error:

$$\mathbb{P}(\text{accept } \mathbf{w}^*) = \min \left(1, \frac{\exp(-H(\mathbf{w}^*, \mathbf{p}^*))}{\exp(-H(\mathbf{w}, \mathbf{p}))} \right)$$

Sampling proceeds as a Gibbs scheme: draw a fresh momentum $\mathbf{p} \sim \mathcal{N}(0, \mathbf{M})$, run L leapfrog steps to propose $(\mathbf{w}^*, \mathbf{p}^*)$, accept/reject via the rule above, repeat.

Recap: Hamiltonian Monte Carlo (HMC) and NUTS V

The Tuning Problem, and How NUTS Solves It

HMC has two tunable knobs: step size ϵ and trajectory length L (number of leapfrog steps). Too short a trajectory behaves like a random walk; too long wastes computation retracing its own path. Both step size and trajectory length traditionally require expensive pilot runs to tune. The **No-U-Turn Sampler (NUTS)** automates both:

- **Step size** is tuned automatically during burn-in via *dual averaging* to hit a target acceptance rate δ .
- **Trajectory length** is set by doubling the simulated path until the trajectory starts to double back on itself (a “U-turn”, detected via $(\mathbf{w}^* - \mathbf{w}) \cdot \mathbf{p} \leq 0$) or the Hamiltonian diverges. A slice-sampling correction repairs detailed balance, which this doubling procedure would otherwise break.

NUTS performs at least as well as a well-tuned HMC, without requiring costly pilot runs – making it far more accessible to non-expert users.

Recap: Hamiltonian Monte Carlo (HMC) and NUTS VI

Dual Averaging (Eqs. 11.6-11.7)

$$\epsilon_{t+1} \leftarrow \mu - \frac{\sqrt{t}}{\gamma} \frac{1}{t + t_0} \sum_{i=1}^t H_i, \quad \bar{\epsilon}_{t+1} \leftarrow \eta_t \epsilon_{t+1} + (1 - \eta_t) \bar{\epsilon}_t$$

where η_t is a decaying adaptation rate, γ regulates convergence towards μ , $H_t = \delta - \alpha_t$ is the (running) gap between the target and the actual acceptance rate ($\mathbb{E}[H_t] = 0$ at convergence), and $\mu = \log(10\epsilon_0)$ is a free parameter that ϵ_t gravitates to. Typical settings: $\bar{\epsilon}_0 = 1$, $\bar{H}_0 = 0$, $\gamma = 0.05$, $t_0 = 10$, $\kappa = 0.75$; the book uses $\epsilon_0 = 0.0001$ and targets an 80% acceptance rate.

Python: Recap – Plain HMC Leapfrog Step

Listing 2: One HMC transition, from scratch

```
1 import numpy as np
2
3 def leapfrog(w, p, eps, L, grad_U):
4     """Eq. 11.4: L leapfrog steps, mass M=I."""
5     p = p - 0.5 * eps * grad_U(w)
6     for step in range(L):
7         w = w + eps * p
8         if step != L - 1:
9             p = p - eps * grad_U(w)
10    p = p - 0.5 * eps * grad_U(w)
11    return w, -p    # negate p: time-reversibility
12
13 def hmc_step(w, U, grad_U, eps, L, rng):
14     """One Metropolis-corrected HMC move."""
15     p0 = rng.normal(size=w.shape)
16     H0 = U(w) + 0.5 * np.sum(p0**2)
17     w_new, p_new = leapfrog(w, p0, eps, L, grad_U)
18     H1 = U(w_new) + 0.5 * np.sum(p_new**2)
19     log_alpha = -(H1 - H0)    # Eq. 11.5
20     accept = np.log(rng.uniform()) < log_alpha
21     return (w_new if accept else w), accept
```

Reading the Code

$U(w)$ is the potential energy: minus the log posterior of the NS parameters $w = (\beta_0, \beta_1, \beta_2, \lambda)$ given yield data. grad_U is its gradient. `leapfrog` implements Eq. 11.4 exactly; `hmc_step` implements the Gibbs scheme: refresh momentum, integrate, Metropolis-correct (Eq. 11.5).

Used Below

This exact pattern (with w reparametrized so $\beta_0, \lambda > 0$) is what powers the toy calibration example later in this deck.

S2HMC: Why We Need a “Shadow” Hamiltonian I

Intuition: What’s Actually New Here

Even a well-tuned HMC still produces *autocorrelated* samples, because the leapfrog integrator only conserves the true Hamiltonian H up to $\mathcal{O}(\epsilon^2)$ — the energy error compounds over the trajectory, inflating the change in H used in the Metropolis step and causing more rejections. Two fixes exist: (i) use a higher-order (more accurate) integrator, or (ii) restrict to tiny step sizes / short trajectories. Both cost more compute. **Separable Shadow Hamiltonian Hybrid Monte Carlo (S2HMC)** takes a third, cleverer route: instead of trying to conserve the true H more accurately, it derives a nearby, modified “*shadow*” Hamiltonian $\tilde{H}$ that the leapfrog integrator conserves *almost exactly*, then corrects the resulting bias afterwards with importance weights.

S2HMC: Why We Need a “Shadow” Hamiltonian II

The Shadow Hamiltonian Expansion (Eqs. 11.8-11.9)

Backward error analysis of the leapfrog integrator shows it exactly conserves an asymptotic expansion in powers of the step size ϵ :

$$\tilde{H} = H + \epsilon H_2 + \epsilon^2 H_3 + \epsilon^3 H_4 + \dots$$

To keep this practical, we truncate at some order k :

$$\tilde{H}^{[k]} = H + \epsilon H_2 + \epsilon^2 H_3 + \epsilon^3 H_4 + \dots = \tilde{H} + \mathcal{O}(\epsilon^k)$$

For symplectic integrators like leapfrog, these shadow Hamiltonians are themselves genuine Hamiltonian systems, so we can simulate them just like the original.

S2HMC: Why We Need a “Shadow” Hamiltonian III

Fourth-Order Truncation (Eq. 11.10)

This chapter uses a 4th-order truncation, conserved to $\mathcal{O}(\epsilon^4)$ – more accurate than the true Hamiltonian itself under 2nd-order-accurate leapfrog:

$$\tilde{H}^{[4]} = U(\mathbf{w}) + K(\mathbf{p}) + \frac{\epsilon^2}{12} \mathbf{K}_{\mathbf{p}}^T \mathbf{U}_{\mathbf{w}\mathbf{w}} \mathbf{K}_{\mathbf{p}} - \frac{\epsilon^2}{24} \mathbf{U}_{\mathbf{w}}^T \mathbf{K}_{\mathbf{p}\mathbf{p}} \mathbf{U}_{\mathbf{w}} + \mathcal{O}(\epsilon^4)$$

where $\mathbf{U}_{\mathbf{w}}$, $\mathbf{U}_{\mathbf{w}\mathbf{w}}$, $\mathbf{K}_{\mathbf{p}}$, $\mathbf{K}_{\mathbf{p}\mathbf{p}}$ are Jacobians and Hessians of the potential and kinetic energies. The problem: this $\tilde{H}^{[4]}$ is **non-separable** in $\mathbf{w}$ and $\mathbf{p}$ (the cross term mixes them), which is expensive – it needs a Hessian $\mathbf{U}_{\mathbf{w}\mathbf{w}}$ and a costly momentum acceptance criterion.

S2HMC's Trick: Make It Separable

S2HMC (Sweet et al., 2009) pre- and post-processes the leapfrog integrator with a change of variables so that the *effective* shadow Hamiltonian becomes separable – i.e. splits cleanly into a position-only part and a momentum-only part, which is exactly what makes leapfrog cheap and simple in ordinary HMC.

S2HMC: Why We Need a “Shadow” Hamiltonian IV

The Separable Shadow Hamiltonian (Eq. 11.11)

$$\tilde{H}(\mathbf{w}, \mathbf{p}) = U(\mathbf{w}) + K(\mathbf{p}) + \frac{\epsilon^2}{24} \mathbf{U}_{\mathbf{w}}^T \mathbf{M}^{-1} \mathbf{U}_{\mathbf{w}} + \mathcal{O}(\epsilon^4)$$

Only a gradient $\mathbf{U}_{\mathbf{w}}$ is needed now (no Hessian!) – much cheaper than Eq. 11.10.

Pre-Processing: The Reversible Mapping (Eqs. 11.12-11.13)

Before running leapfrog, positions and momenta are mapped via $(\hat{\mathbf{w}}, \hat{\mathbf{p}}) = \mathcal{X}(\mathbf{w}, \mathbf{p})$, solved by fixed-point iteration:

$$\begin{aligned}\hat{\mathbf{p}} &= \mathbf{p} - \frac{\epsilon}{24} \left[\mathbf{U}_{\mathbf{w}}(\mathbf{w} + \epsilon \mathbf{M}^{-1} \hat{\mathbf{p}}) - \mathbf{U}_{\mathbf{w}}(\mathbf{w} - \epsilon \mathbf{M}^{-1} \hat{\mathbf{p}}) \right] \\ \hat{\mathbf{w}} &= \mathbf{w} + \frac{\epsilon^2 \mathbf{M}^{-1}}{24} \left[\mathbf{U}_{\mathbf{w}}(\mathbf{w} + \epsilon \mathbf{M}^{-1} \hat{\mathbf{p}}) + \mathbf{U}_{\mathbf{w}}(\mathbf{w} - \epsilon \mathbf{M}^{-1} \hat{\mathbf{p}}) \right]\end{aligned}$$

S2HMC: Why We Need a “Shadow” Hamiltonian V

Post-Processing: Reversing the Mapping (Eqs. 11.14-11.15)

After ordinary leapfrog is run on $(\hat{\mathbf{w}}, \hat{\mathbf{p}})$, the mapping is undone:

$$\begin{aligned}\mathbf{w} &= \hat{\mathbf{w}} - \frac{\epsilon^2 \mathbf{M}^{-1}}{24} \left[\mathbf{U}_{\mathbf{w}}(\mathbf{w} + \epsilon \mathbf{M}^{-1} \hat{\mathbf{p}}) + \mathbf{U}_{\mathbf{w}}(\mathbf{w} - \epsilon \mathbf{M}^{-1} \hat{\mathbf{p}}) \right] \\ \mathbf{p} &= \hat{\mathbf{p}} + \frac{\epsilon}{24} \left[\mathbf{U}_{\mathbf{w}}(\mathbf{w} + \epsilon \mathbf{M}^{-1} \hat{\mathbf{p}}) - \mathbf{U}_{\mathbf{w}}(\mathbf{w} - \epsilon \mathbf{M}^{-1} \hat{\mathbf{p}}) \right]\end{aligned}$$

Correcting the Bias: Importance Weights

Because we are now sampling from the shadow density (built from $\tilde{H}$) rather than the true target density (built from H), **importance weights** are computed *after* the samples are drawn to correct back to the true posterior. As a sanity check: S2HMC *degenerates exactly to ordinary HMC* when the shadow Hamiltonian is set equal to the true Hamiltonian ($\tilde{H} = H$). In this chapter's experiments, HMC and S2HMC both use a fixed trajectory length $L = 25$, denoted HMC_{25} and S2HMC_{25} .

Python: S2HMC Fixed-Point Pre-Processing Map (Eq. 11.12-11.13)

Listing 3: S2HMC reversible map, illustrative

```
1 import numpy as np
2
3 def s2hmc_preprocess(w, p, eps, grad_U,
4                     M_inv=1.0, n_fp=6):
5     """Fixed-point solve for (what, phat),
6     Eqs. 11.12-11.13, separable S2HMC."""
7     what, phat = w.copy(), p.copy()
8     for _ in range(n_fp):
9         gplus = grad_U(w + eps * M_inv * phat)
10        gminus = grad_U(w - eps * M_inv * phat)
11        phat = p - (eps / 24) * (gplus - gminus)
12        what = w + (eps**2 * M_inv / 24) * (
13            gplus + gminus)
14    return what, phat
15
16 # S2HMC step: preprocess -> ordinary leapfrog
17 # on (what, phat) -> postprocess (Eq. 11.14-15)
18 # -> importance-weight the accepted sample.
```

Why Fixed-Point Iteration

$\hat{\mathbf{p}}$ and $\hat{\mathbf{w}}$ appear on *both* sides of Eqs. 11.12-11.13 (implicit equations), so we solve them by iterating a handful of times starting from $\hat{\mathbf{p}}_0 = \mathbf{p}$, $\hat{\mathbf{w}}_0 = \mathbf{w}$ – this converges quickly since the correction term is $\mathcal{O}(\epsilon)$ / $\mathcal{O}(\epsilon^2)$, i.e. small.

Cost vs. Benefit

This pre/post-processing costs a handful of extra gradient evaluations per leapfrog trajectory, in exchange for far better energy conservation and (after importance-weighting) lower autocorrelation than plain HMC at the same step size.

Outline

- 1 11.1 Introduction
- 2 11.2 The Nelson-Siegel Model
- 3 11.3 Advanced Hamiltonian Monte Carlo Methods
 - 11.3.1 HMC and the No-U-Turn Sampler
 - 11.3.2 Separable Shadow Hamiltonian Hybrid Monte Carlo
- 4 **Toy Illustration: Bayesian NS Calibration with HMC**
- 5 11.4 Numerical Experiments
 - 11.4.1 Data Description and Experiment Setup
 - 11.4.2 Performance Metrics
- 6 11.5 Results and Discussions
- 7 11.6 Conclusion

Toy Running Example: Recovering NS Parameters I

This Is Our Own Illustrative Simulation

Everything on this and the next slide is a **toy simulation built for this slide deck**, written independently from scratch in `figures/gen_figures.py`. It is *not* the book's own experiment code, though we deliberately reuse the same true parameter values the book quotes for its simulated experiment (Sec. 11.4.1) so the story can be sanity-checked against the book's qualitative conclusions. The book's own numerical results are summarized separately in Sec. 11.5 below.

Step 1: Simulate a Synthetic Yield Curve

- True NS parameters: $\beta_0 = 0.04$, $\beta_1 = -0.05$, $\beta_2 = 0.20$, $\lambda = 1.3$ (same values as the book's Sec. 11.4.1 simulated experiment).
- Maturities (years): 0.25, 0.5, 1, 2, 3, 5, 7, 10, 15, 20, 30.
- Add i.i.d. Gaussian noise with standard deviation 0.05 percentage points (5 basis points) to each true yield to mimic market quote noise.

Toy Running Example: Recovering NS Parameters II

Step 2: Bayesian Calibration via From-Scratch HMC

We reparametrize $\beta_0 = e^{z_0}$, $\lambda = e^{z_3}$ so positivity is automatic (implementing the book's LogNormal priors as Gaussian priors on z_0, z_3), keep Gaussian priors directly on β_1, β_2 , and run the plain HMC sampler from the previous section (step size $\epsilon = 10^{-4}$, trajectory length $L = 25$ – matching the book's fixed $L = 25$ for HMC/S2HMC) for 4,000 post-burn-in samples (1,000 burn-in), initialized *away* from the truth to demonstrate recovery.

Toy Running Example: Recovering NS Parameters III

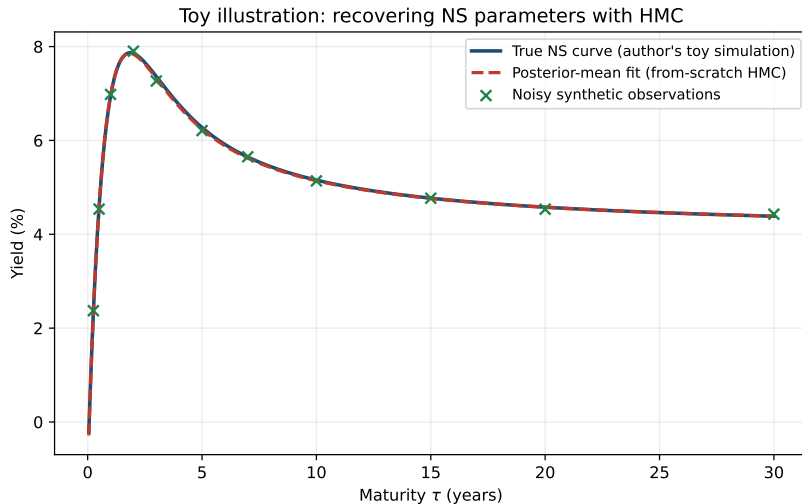
Result: Recovered Parameters vs. Truth

Our from-scratch HMC sampler achieved an acceptance rate of **99.5%**. Posterior means (with posterior standard deviations) compare to the true values as follows:

Parameter	True value	Posterior mean	Posterior std. dev.
β_0	0.0400	0.0401	0.0003
β_1	-0.0500	-0.0510	0.0012
β_2	0.2000	0.2003	0.0021
λ	1.3000	1.3232	0.0193

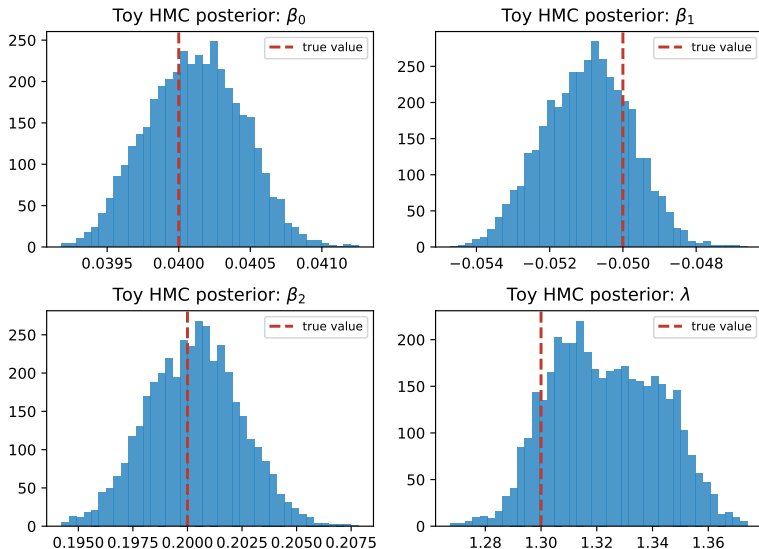
All four parameters are recovered to within about one posterior standard deviation of the truth, and the fitted curve matches the true curve to a root-mean-square error of only **0.021** basis points across the 11 tenors – consistent with the book's own qualitative finding (Table 11.1) that HMC, S2HMC, and NUTS can all reproduce the true parameters that generated simulated yield curve data.

Figure: Toy Calibration – True Curve vs. HMC-Recovered Fit



Own toy simulation: true NS curve (solid), noisy synthetic observations (markers), posterior-mean fit from our from-scratch HMC sampler (dashed) – visually indistinguishable from the truth.

Figure: Toy Posterior Distributions of the NS Parameters



Outline

- 1 11.1 Introduction
- 2 11.2 The Nelson-Siegel Model
- 3 11.3 Advanced Hamiltonian Monte Carlo Methods
 - 11.3.1 HMC and the No-U-Turn Sampler
 - 11.3.2 Separable Shadow Hamiltonian Hybrid Monte Carlo
- 4 Toy Illustration: Bayesian NS Calibration with HMC
- 5 11.4 Numerical Experiments
 - 11.4.1 Data Description and Experiment Setup
 - 11.4.2 Performance Metrics
- 6 11.5 Results and Discussions
- 7 11.6 Conclusion

The Book's Data and Experimental Setup I

Three Datasets

- 1 **Simulated data:** generated from the NS model with *known* parameters $\{\beta_0 = 0.04, \beta_1 = -0.05, \beta_2 = 0.2, \lambda = 1.3\}$, so ground truth is available to check whether the MCMC methods can recover it.
- 2 **South African Rand (ZAR) Swap Curve** as at 29 March 2024 – a single real-world cross-section.
- 3 **USA Corporate Bond yield curve time series**, 2018–2024 – to see how calibrated parameters evolve through time (including through the COVID-19 period).

Methods Compared

OLS (the traditional baseline) versus three Bayesian MCMC samplers: **HMC**, **S2HMC**, and **NUTS**.

The Book's Data and Experimental Setup II

Priors and MCMC Settings

- Priors: **Gaussian** for β_1, β_2 ; **LogNormal** for β_0, λ (enforcing $\beta_0, \lambda > 0$).
- Pre-conditioning / mass matrix $\mathbf{M} = \mathbf{I}$ for all MCMC methods (the common default in practice).
- **5 chains**, each with **2,000** post-burn-in samples and a burn-in of **1,000** – sufficient for convergence on every dataset tested.
- Number of leapfrog steps fixed at $L = 25$ for HMC and S2HMC (hence HMC_{25} , S2HMC_{25} in the tables).
- Step sizes chosen by targeting an **80% acceptance rate** via the dual-averaging method (Eqs. 11.6-11.7).

Performance Metrics

Predictive Performance on Unseen (Held-Out) Data

All methods are compared on their ability to predict yields *not* used for calibration, using four standard metrics:

- **MSE** – Mean Square Error.
- **MAE** – Mean Absolute Error.
- R^2 – coefficient of determination (fraction of variance explained).
- **MAPE** – Mean Absolute Percentage Error.

Why Predictive, Not Just In-Sample, Performance?

Because collinearity among the NS parameters means a model can fit the *calibration* data almost perfectly while still being unstable or poorly generalizing to nearby maturities – exactly the instability problem that motivated moving away from plain OLS in the first place. Testing on held-out points is the fairer comparison.

Outline

- 1 11.1 Introduction
- 2 11.2 The Nelson-Siegel Model
- 3 11.3 Advanced Hamiltonian Monte Carlo Methods
 - 11.3.1 HMC and the No-U-Turn Sampler
 - 11.3.2 Separable Shadow Hamiltonian Hybrid Monte Carlo
- 4 Toy Illustration: Bayesian NS Calibration with HMC
- 5 11.4 Numerical Experiments
 - 11.4.1 Data Description and Experiment Setup
 - 11.4.2 Performance Metrics
- 6 11.5 Results and Discussions
- 7 11.6 Conclusion

Results on Simulated Yield Curve Data I

Table 11.1: Calibrated Parameters vs. Truth (Simulated Data)

Parameter	True	NUTS	HMC ₂₅	S2HMC ₂₅
β_0	0.040	0.040	0.040	0.040
β_1	-0.050	-0.050	-0.050	-0.051
β_2	0.200	0.200	0.200	0.200
λ	1.300	1.295	1.295	1.292

All three MCMC methods reproduce the true parameters that generated the simulated curve, essentially indistinguishably from one another.

Results on Simulated Yield Curve Data II

Table 11.2: Predictive Performance (Simulated Data)

Metric	NUTS	S2HMC ₂₅	HMC ₂₅
R^2	0.999988	0.99998	0.999992
MSE	2.925e-05	4.420e-05	1.871e-05
MAPE	0.000854	0.001092	0.000718
MAE	0.004398	0.005338	0.003197

All three methods perform similarly well on held-out simulated data, with plain HMC achieving the overall best predictive numbers here.

Results on Simulated Yield Curve Data III

Table 11.3: ZAR Swap Curve – OLS vs. NUTS (29 March 2024)

Parameter	OLS	NUTS
β_0	0.1304	0.1311
β_1	-0.7444	-0.0456
β_2	-0.0812	-0.0810
λ	2.5599	2.6179

β_0 and β_2 agree closely between OLS and NUTS, but β_1 (the slope factor) diverges sharply – a symptom of the collinearity and instability that motivated the Bayesian, regularized approach in the first place.

Results on Simulated Yield Curve Data IV

Table 11.4: ZAR Swap Curve – Predictive Performance

Metric	OLS	NUTS
R^2	0.993763	0.993787
MSE	0.007017	0.006990
MAPE	0.007804	0.007636
MAE	0.071268	0.070161

NUTS marginally outperforms OLS on *every* metric on this real-world curve, while additionally providing full posterior uncertainty.

Results on Simulated Yield Curve Data V

Qualitative Findings from the Posterior Distributions

Figures 11.3-11.5 in the book (calibrated parameter posteriors from HMC, S2HMC, and NUTS on the simulated data) show that the three MCMC algorithms produce visually near-identical posterior distributions across all four parameters – reassuring evidence that the choice of sampler does not materially change the substantive conclusions here, only the compute cost and sample efficiency.

Results on Simulated Yield Curve Data VI

Time-Varying Calibration: USA Corporate Bonds, 2018-2024

Calibrating the NS model separately at each date in the USA corporate bond time series (using NUTS and OLS, with 1.5 standard-deviation NUTS confidence bands) reveals:

- β_0 and β_2 estimates from OLS and NUTS agree closely, with narrow NUTS confidence bands – high confidence in these parameters.
- β_1 (the slope estimates) diverge early in the sample but later align; confidence bands are wider here, and OLS estimates sometimes fall *outside* the NUTS confidence bands, with more extreme OLS movements through time.
- λ estimates from OLS and NUTS broadly align; large upward moves in λ widen the upper confidence band.
- The out-of-sample MSE for both NUTS and OLS was near zero before the COVID-19 pandemic (2020), stayed elevated through late 2023, and reverted back towards zero afterwards – tracking the broader market disruption of that period.

Outline

- 1 11.1 Introduction
- 2 11.2 The Nelson-Siegel Model
- 3 11.3 Advanced Hamiltonian Monte Carlo Methods
 - 11.3.1 HMC and the No-U-Turn Sampler
 - 11.3.2 Separable Shadow Hamiltonian Hybrid Monte Carlo
- 4 Toy Illustration: Bayesian NS Calibration with HMC
- 5 11.4 Numerical Experiments
 - 11.4.1 Data Description and Experiment Setup
 - 11.4.2 Performance Metrics
- 6 11.5 Results and Discussions
- 7 11.6 Conclusion

Conclusion and Outlook I

What This Chapter Showed

This chapter presented a **first-in-the-literature Bayesian framework** for calibrating the Nelson-Siegel model to real (ZAR swap curve, USA corporate bond curves) and simulated yield curve data, using three advanced Hamiltonian-dynamics MCMC samplers: **HMC**, **Separable Shadow Hybrid HMC (S2HMC)**, and the **No-U-Turn Sampler (NUTS)**.

Key Takeaways

- The Bayesian MCMC framework produces results similar to or *better than* traditional OLS-based calibration across multiple predictive performance metrics.
- It additionally yields **full posterior distributions** over the NS parameters, giving honest uncertainty quantification that OLS point estimates cannot provide – essential for understanding parameter movements through time and building confidence intervals on predictions.
- S2HMC and NUTS both offer practical alternatives to hand-tuned HMC, at essentially no cost to accuracy on the datasets studied.

Future Work (as noted by the authors)

Extending this Bayesian MCMC framework to the **dynamic** NS model (capturing the yield curve's evolution through time within a single model), and to modeling the **term structure of volatilities**, where initial studies already show promise. The next chapter in the book turns to **yield curve model selection** via the Nested Sampling approach of Skilling (2004) and its extensions.